

Deep Learning as Function Composition

From Neurons and Functions to CNN, ResNet, Attention, and Vision Transformers

Tutorial for undergraduate students

Tóm tắt nội dung

This tutorial develops a mathematical and programming-oriented understanding of convolutional neural networks (CNNs). Instead of memorizing architectures, students learn to interpret deep learning as the composition of functions. The tutorial starts from a neuron and linear function, introduces nonlinear activation functions, explains layers and composition, and then derives the main operations of CNNs: convolution, stride, pooling, normalization, flattening, and classification. The tutorial then explains the evolution from LeNet and AlexNet to VGG, Inception, ResNet, DenseNet, U-Net, MobileNet, EfficientNet, attention-based CNNs, and Vision Transformers. Each major idea is connected to executable Python/PyTorch code.

Mục lục

1	Learning Objectives	4
2	The Mathematical Starting Point: A Function	4
3	The Neuron	5
3.1	Linear neuron	5
3.2	Numerical example	5
3.3	The corresponding PyTorch code	5
4	Why Activation Functions Are Necessary	6
5	Important Activation Functions	6
5.1	ReLU	6
5.2	Sigmoid	7
5.3	Tanh	7
5.4	GELU and SiLU	7
6	A Layer Is a Function	7
7	Deep Learning as Function Composition	8
8	From Fully Connected Networks to Images	8
9	Convolution	9
9.1	The basic idea	9
9.2	Simple numerical example	9
9.3	Why convolution is useful	9
9.4	Local connectivity	9
9.5	Weight sharing	10
10	Convolution Output Size	10
10.1	Example	10

11 PyTorch Convolution	10
12 Activation After Convolution	11
13 Pooling	11
14 Stride Convolution	12
15 Batch Normalization	12
16 A Complete Small CNN	13
17 Classifier and Softmax	14
18 Training a CNN	14
19 The Fundamental CNN Representation Hierarchy	15
20 The Evolution of CNN Architectures	15
21 LeNet	16
22 AlexNet	16
23 VGG	16
24 Inception	17
25 Residual Networks	17
25.1 The degradation problem	17
25.2 PyTorch residual block	17
26 DenseNet	18
27 U-Net	18
28 MobileNet	19
29 EfficientNet	19
30 Attention	20
31 Channel Attention: SE Block	20
32 Spatial Attention	21
33 CBAM	21
34 CNN + Attention: Why?	22
35 Vision Transformer	22
36 Self-Attention	23
37 CNN versus Transformer	23
38 A Unified View of CNN Evolution	23

39 Problem–Solution Interpretation	23
40 Reading CNN Code Systematically	24
41 A Complete Educational CNN	25
42 Experiment: Inspecting Shapes	26
43 Parameter Counting	27
44 Exercise: Build a CNN from Scratch	27
45 Exercise: Compare CNN Variants	28
46 A Practical Mental Model	28
47 Final Conceptual Map	28

1 Learning Objectives

After completing this tutorial, a student should be able to:

1. explain a neuron as a mathematical function;
2. explain why nonlinear activation functions are necessary;
3. understand deep learning as function composition;
4. identify the role of a layer in a neural network;
5. calculate the output size of a convolution;
6. explain local connectivity and weight sharing;
7. distinguish convolution, pooling, and strided convolution;
8. read the basic structure of a PyTorch CNN;
9. explain why CNN architectures became deeper and more complex;
10. understand residual connections mathematically and in code;
11. explain the motivation for attention;
12. distinguish local convolutional interaction from global self-attention;
13. recognize the major families of CNN evolution;
14. modify a CNN rather than merely copying an existing architecture.

The central principle is:

Deep Learning = learning a complex function by composing simpler functions.

2 The Mathematical Starting Point: A Function

Before studying neural networks, consider an ordinary function

$$f : \mathbb{R}^d \rightarrow \mathbb{R}^m.$$

For example,

$$f(x) = 2x + 1.$$

If $x = 3$, then

$$f(3) = 7.$$

A neural network is also a function. The difference is that the function is not manually specified. Its parameters are learned from data.

For a parameterized function,

$$f_{\theta}(x),$$

the parameter vector θ is learned during training.

Thus the learning problem can be written as

Find θ such that $f_{\theta}(x)$ produces useful outputs.

3 The Neuron

3.1 Linear neuron

The simplest neuron computes

$$z = \sum_{i=1}^d w_i x_i + b.$$

In vector notation,

$$z = \mathbf{w}^T \mathbf{x} + b.$$

Here

- $\mathbf{x}$ is the input;
- $\mathbf{w}$ is the weight vector;
- b is the bias;
- z is the output before activation.

3.2 Numerical example

Let

$$\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 0.5 \\ -1 \\ 2 \end{bmatrix}, \quad b = 0.5.$$

Then

$$z = 0.5(2) - 1(3) + 2(1) + 0.5.$$

Therefore,

$$z = 1 - 3 + 2 + 0.5 = 0.5.$$

The neuron has implemented a function

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

3.3 The corresponding PyTorch code

```
1 import torch
2 import torch.nn as nn
3
4 x = torch.tensor([[2.0, 3.0, 1.0]])
5
6 neuron = nn.Linear(3, 1)
7
8 with torch.no_grad():
9     neuron.weight[:] = torch.tensor([[0.5, -1.0, 2.0]])
10    neuron.bias[:] = torch.tensor([0.5])
11
12 y = neuron(x)
13
14 print(y)
```

The essential correspondence is

$$\text{nn.Linear}(3,1) \longleftrightarrow \mathbf{w}^T \mathbf{x} + b.$$

4 Why Activation Functions Are Necessary

Suppose we compose two linear functions:

$$f_1(x) = a_1x + b_1$$

and

$$f_2(x) = a_2x + b_2.$$

Then

$$f_2(f_1(x)) = a_2(a_1x + b_1) + b_2.$$

Therefore,

$$f_2(f_1(x)) = (a_2a_1)x + (a_2b_1 + b_2),$$

which is still linear-affine.

Consequently,

$$\text{Linear} \circ \text{Linear} = \text{Linear}.$$

A deep network containing only linear layers would not obtain the expressive power expected from deep learning.

We therefore introduce a nonlinear function

$$h = \sigma(z).$$

5 Important Activation Functions

5.1 ReLU

The Rectified Linear Unit is

$$\text{ReLU}(x) = \max(0, x).$$

For example,

$$\text{ReLU}(-2) = 0, \quad \text{ReLU}(3) = 3.$$

Python:

```
1 import torch
2 import torch.nn.functional as F
3
4 x = torch.tensor([-2.0, -0.5, 0.0, 1.0, 3.0])
5 y = F.relu(x)
6
7 print(y)
```

5.2 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its range is

$$0 < \sigma(x) < 1.$$

It is useful for binary probabilities, although modern hidden layers usually prefer ReLU-family functions.

5.3 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Its range is

$$-1 < \tanh(x) < 1.$$

5.4 GELU and SiLU

Modern architectures often use

$$\text{GELU}(x)$$

or

$$\text{SiLU}(x) = x\sigma(x).$$

The exact choice is architecture-dependent.

6 A Layer Is a Function

A layer should be understood as a function.

For example,

$$f(\mathbf{x}) = \text{ReLU}(W\mathbf{x} + \mathbf{b})$$

is one neural-network layer.

In PyTorch:

```
1 layer = nn.Sequential(
2     nn.Linear(10, 20),
3     nn.ReLU()
4 )
```

Mathematically,

$$f(\mathbf{x}) = \text{ReLU}(W\mathbf{x} + \mathbf{b}).$$

Therefore:

$$\text{Layer} \longleftrightarrow \text{Function}.$$

7 Deep Learning as Function Composition

Consider

$$f_1(\mathbf{x}) = W_1\mathbf{x} + \mathbf{b}_1,$$

$$f_2(\mathbf{h}) = \text{ReLU}(\mathbf{h}),$$

$$f_3(\mathbf{h}) = W_2\mathbf{h} + \mathbf{b}_2.$$

The network is

$$F(\mathbf{x}) = f_3 \circ f_2 \circ f_1(\mathbf{x}).$$

Explicitly,

$$F(\mathbf{x}) = W_2 \text{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2.$$

This is the fundamental mathematical idea behind deep learning.

8 From Fully Connected Networks to Images

An image is not merely a vector. A color image can be represented as

$$X \in \mathbb{R}^{C \times H \times W},$$

where

- C = number of channels;
- H = height;
- W = width.

For RGB images,

$$C = 3.$$

For an image of size 224×224 ,

$$X \in \mathbb{R}^{3 \times 224 \times 224}.$$

Flattening gives

$$3 \cdot 224 \cdot 224 = 150528$$

numbers.

A fully connected layer from this vector to 1000 outputs requires approximately

$$150528 \times 1000$$

weights, i.e. more than 150 million parameters.

This motivates a more suitable operation.

Convolution

9 Convolution

9.1 The basic idea

A convolution kernel is a small matrix.

For example,

$$K = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{bmatrix}.$$

At each image location, the kernel interacts with a local image patch.

For a single channel,

$$Y(i, j) = \sum_{u=1}^k \sum_{v=1}^k K(u, v) X(i + u, j + v).$$

The output Y is called a feature map.

9.2 Simple numerical example

Let

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

The first output is

$$Y(1, 1) = 1(1) + 0(2) + 0(4) - 1(5) = -4.$$

Thus convolution is a local weighted sum.

9.3 Why convolution is useful

Three ideas are fundamental:

1. local connectivity;
2. weight sharing;
3. spatially structured computation.

9.4 Local connectivity

A convolution neuron sees only a small neighborhood.

For a 3×3 kernel:

$\text{receptive field} = 3 \times 3$

at that layer.

9.5 Weight sharing

The same kernel is used at many positions.

Thus the network does not learn a separate detector for every pixel location.

This greatly reduces the number of parameters.

10 Convolution Output Size

For one spatial dimension,

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1,$$

where

- H = input size;
- K = kernel size;
- P = padding;
- S = stride.

For a square image,

$$W_{\text{out}} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1.$$

10.1 Example

Let

$$H = W = 32, \quad K = 3, \quad P = 1, \quad S = 1.$$

Then

$$H_{\text{out}} = \frac{32 + 2 - 3}{1} + 1 = 32.$$

Thus a 3×3 convolution with padding 1 and stride 1 preserves spatial resolution.

11 PyTorch Convolution

```
1 import torch
2 import torch.nn as nn
3
4 conv = nn.Conv2d(
5     in_channels=3,
6     out_channels=16,
7     kernel_size=3,
8     stride=1,
9     padding=1
10 )
11
12 x = torch.randn(8, 3, 32, 32)
13 y = conv(x)
14
15 print(x.shape)
16 print(y.shape)
```

The tensor shapes are

$$(8, 3, 32, 32) \rightarrow (8, 16, 32, 32).$$

The first dimension is batch size.

The convolution has learned

$$16$$

kernels, each operating on the three input channels.

The parameter count is

$$16(3)(3)(3) + 16 = 448.$$

12 Activation After Convolution

A typical CNN block is

$$X \rightarrow \text{Conv} \rightarrow \text{ReLU}.$$

Mathematically,

$$Y = \text{ReLU}(K * X + b).$$

In PyTorch:

```
1 block = nn.Sequential(
2     nn.Conv2d(3, 16, kernel_size=3, padding=1),
3     nn.ReLU()
4 )
```

This is again function composition.

13 Pooling

Max pooling is defined by

$$Y(i, j) = \max_{(u, v) \in \mathcal{N}(i, j)} X(i + u, j + v).$$

A common choice is

$$2 \times 2$$

pooling with stride 2.

Therefore,

$$32 \times 32 \rightarrow 16 \times 16.$$

PyTorch:

```
1 pool = nn.MaxPool2d(kernel_size=2, stride=2)
2
3 x = torch.randn(8, 16, 32, 32)
4 y = pool(x)
5
6 print(y.shape)
```

Pooling mainly provides

spatial reduction

and increases the effective receptive field of later layers.

14 Stride Convolution

Instead of

$$Conv \rightarrow Pool,$$

a network can use

$$Conv(\text{stride} = 2).$$

Example:

```
1 downsample = nn.Conv2d(  
2     16, 32,  
3     kernel_size=3,  
4     stride=2,  
5     padding=1  
6 )
```

This changes

$$32 \times 32 \rightarrow 16 \times 16.$$

Modern architectures frequently use strided convolutions for learned downsampling.

15 Batch Normalization

Batch normalization transforms activations using a batch-dependent normalization followed by learnable scale and shift.

A simplified expression is

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

followed by

$$y = \gamma \hat{x} + \beta.$$

PyTorch:

```
1 block = nn.Sequential(  
2     nn.Conv2d(3, 32, 3, padding=1),  
3     nn.BatchNorm2d(32),  
4     nn.ReLU()  
5 )
```

The exact placement of normalization varies between architectures.

16 A Complete Small CNN

A basic image classifier can be written as

$$\begin{aligned} X &\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ &\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ &\rightarrow \text{Flatten} \rightarrow \text{Linear}. \end{aligned}$$

PyTorch implementation:

```

1 import torch
2 import torch.nn as nn
3
4 class SmallCNN(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7
8         self.features = nn.Sequential(
9             nn.Conv2d(3, 32, 3, padding=1),
10             nn.ReLU(),
11             nn.MaxPool2d(2),
12
13             nn.Conv2d(32, 64, 3, padding=1),
14             nn.ReLU(),
15             nn.MaxPool2d(2)
16         )
17
18         self.classifier = nn.Sequential(
19             nn.Flatten(),
20             nn.Linear(64 * 8 * 8, 128),
21             nn.ReLU(),
22             nn.Linear(128, num_classes)
23         )
24
25     def forward(self, x):
26         x = self.features(x)
27         x = self.classifier(x)
28         return x
29
30 model = SmallCNN(num_classes=10)
31
32 x = torch.randn(4, 3, 32, 32)
33 logits = model(x)
34
35 print(logits.shape)

```

The shape progression is

$$3 \times 32 \times 32$$

$\downarrow \text{Conv}$

$$32 \times 32 \times 32$$

$\downarrow \text{Pool}$

$$32 \times 16 \times 16$$

$\downarrow \textit{Conv}$

$$64 \times 16 \times 16$$

$\downarrow \textit{Pool}$

$$64 \times 8 \times 8.$$

Then

$$64 \cdot 8 \cdot 8 = 4096$$

features are flattened.

17 Classifier and Softmax

A classifier produces logits

$$\mathbf{z} = [z_1, \dots, z_C].$$

Softmax converts logits into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}.$$

Therefore,

$$\sum_i p_i = 1.$$

In PyTorch, when using cross entropy loss, it is normally preferable to pass raw logits:

```

1 criterion = nn.CrossEntropyLoss()
2
3 logits = model(x)
4 loss = criterion(logits, labels)

```

Do not manually apply softmax before `CrossEntropyLoss` in the usual classification setup.

18 Training a CNN

Training consists of four major operations:

forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update.

Example:

```

1 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
2 criterion = nn.CrossEntropyLoss()
3
4 model.train()
5
6 for images, labels in train_loader:
7
8     optimizer.zero_grad()
9
10    logits = model(images)
11
12    loss = criterion(logits, labels)
13
14    loss.backward()
15
16    optimizer.step()

```

Mathematically,

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).$$

Thus training is an optimization problem.

19 The Fundamental CNN Representation Hierarchy

A useful conceptual interpretation is

pixels $\rightarrow$ edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects $\rightarrow$ class.

This should not be interpreted as an exact guarantee for every CNN. Rather, it is a useful conceptual model for understanding hierarchical feature learning.

20 The Evolution of CNN Architectures

The evolution can be understood as a sequence of problems and solutions.

Model/family	Main problem	Main idea
LeNet	Spatial image structure	Convolution and pooling
AlexNet	Large-scale deep learning	ReLU, GPU training, dropout, augmentation
VGG	Need for stronger representation	Simple and deep stacks of 3×3 convolutions
Inception	Multiple object scales	Parallel multi-scale branches
ResNet	Very deep optimization	Residual connection $y = F(x) + x$
DenseNet	Information propagation	Dense feature reuse
U-Net	Dense prediction	Encoder-decoder with skip connections
MobileNet	Computational cost	Depthwise separable convolution
EfficientNet	Scaling CNNs	Compound scaling
SE-Net	Channel importance	Channel attention/gating
CBAM	Channel and spatial importance	Channel + spatial attention

CNN– Transformer ViT	Local/global tradeoff Global interaction	Hybrid convolution and attention Image patches + self-attention
----------------------------	---	--

21 LeNet

LeNet established the basic CNN pattern:

$$Conv \rightarrow Activation \rightarrow Pooling \rightarrow Conv \rightarrow Pooling \rightarrow FC.$$

Its importance is conceptual:

learn spatial features directly from images.

22 AlexNet

AlexNet demonstrated that a deeper CNN trained at large scale could dramatically improve image classification.

Important ideas include

- ReLU activation;
- GPU acceleration;
- dropout;
- data augmentation;
- deeper convolutional representation.

The major lesson is

CNN + large data + computation + nonlinear depth

can produce highly effective learned visual representations.

23 VGG

VGG emphasized architectural simplicity.

Instead of very large kernels, it used repeated

$$3 \times 3$$

convolutions.

For example,

$$7 \times 7$$

can conceptually be replaced by a sequence of smaller convolutions.

The architecture becomes

$$Conv_{3 \times 3} \rightarrow ReLU \rightarrow Conv_{3 \times 3} \rightarrow ReLU.$$

The important lesson is

depth + small local operations.

24 Inception

A single kernel size may not be optimal for every visual structure.

Inception therefore combines several operations:

$$\begin{array}{c} 1 \times 1 \\ 3 \times 3 \\ 5 \times 5 \\ \text{Pooling} \end{array}$$

in parallel, followed by concatenation.

Conceptually,

$$Y = \text{Concat}(f_{1 \times 1}(X), f_{3 \times 3}(X), f_{5 \times 5}(X), f_{\text{pool}}(X)).$$

The problem being addressed is

multi-scale representation.

25 Residual Networks

25.1 The degradation problem

Increasing depth does not automatically guarantee better optimization.

A residual block changes the learning target.

Instead of learning

$$H(X),$$

the block learns

$$F(X) = H(X) - X.$$

Then

$$Y = F(X) + X.$$

The identity path allows information and gradients to travel more directly.

25.2 PyTorch residual block

```

1 class ResidualBlock(nn.Module):
2     def __init__(self, channels):
3         super().__init__()
4
5         self.conv1 = nn.Conv2d(
6             channels, channels, 3, padding=1
7         )
8         self.bn1 = nn.BatchNorm2d(channels)
9
10        self.conv2 = nn.Conv2d(
11            channels, channels, 3, padding=1
12        )
13        self.bn2 = nn.BatchNorm2d(channels)
14
15        self.relu = nn.ReLU()
16

```

```

17 def forward(self, x):
18     identity = x
19
20     out = self.conv1(x)
21     out = self.bn1(out)
22     out = self.relu(out)
23
24     out = self.conv2(out)
25     out = self.bn2(out)
26
27     out = out + identity
28     out = self.relu(out)
29
30     return out

```

The key line is

```

1 out = out + identity

```

which implements

$$Y = F(X) + X.$$

26 DenseNet

DenseNet uses feature concatenation:

$$X_l = H_l([X_0, X_1, \dots, X_{l-1}]).$$

The square brackets denote concatenation.

The major idea is

feature reuse.

Residual networks use addition:

$$F(X) + X.$$

Dense networks use concatenation:

$$[X, F(X)].$$

This is an important architectural distinction.

27 U-Net

U-Net addresses dense prediction.

Classification asks

$$Image \rightarrow Class.$$

Segmentation asks

$$Image \rightarrow \text{label for every pixel}.$$

U-Net uses an encoder-decoder architecture:

$$Image \rightarrow Encoder \rightarrow Bottleneck \rightarrow Decoder \rightarrow Segmentation.$$

Skip connections transfer high-resolution spatial information from encoder layers to decoder layers.

The important idea is

combine semantic information with spatial detail.

28 MobileNet

A standard convolution performs spatial and channel mixing together.

Depthwise separable convolution separates the two operations.

First, depthwise convolution:

$X \rightarrow$ spatial filtering.

Then pointwise convolution:

1×1

to mix channels.

Conceptually,

Depthwise Conv + Pointwise Conv.

This substantially reduces computation.

PyTorch:

```

1 depthwise = nn.Conv2d(
2     32, 32,
3     kernel_size=3,
4     padding=1,
5     groups=32
6 )
7
8 pointwise = nn.Conv2d(
9     32, 64,
10    kernel_size=1
11 )

```

The condition

`groups=32`

makes the first convolution depthwise for 32 input channels.

29 EfficientNet

CNN performance depends on

- depth;
- width;
- input resolution.

Naively increasing only one may be inefficient.
 EfficientNet introduced a principled compound scaling strategy.
 Conceptually,

$$\text{depth} \uparrow, \quad \text{width} \uparrow, \quad \text{resolution} \uparrow$$

are scaled together according to a controlled rule.

The important lesson is

architecture scaling is itself an optimization problem.

30 Attention

CNNs apply local filters. But not all features are equally important.

Attention introduces a mechanism for assigning importance.

A simplified weighted aggregation is

$$Y_i = \sum_j a_{ij} X_j,$$

where

$$a_{ij} \geq 0, \quad \sum_j a_{ij} = 1.$$

The weights a_{ij} determine how strongly one feature interacts with another.

Thus

attention = learned selective interaction.

31 Channel Attention: SE Block

Suppose

$$X \in \mathbb{R}^{C \times H \times W}.$$

A channel attention mechanism produces

$$s \in \mathbb{R}^C.$$

Then

$$Y_c = s_c X_c.$$

A simplified SE block can be implemented as

```

1 class SEBlock(nn.Module):
2     def __init__(self, channels, reduction=4):
3         super().__init__()
4
5         hidden = max(channels // reduction, 1)
6
7         self.pool = nn.AdaptiveAvgPool2d(1)
8
9         self.fc = nn.Sequential(
10             nn.Linear(channels, hidden),
11             nn.ReLU(),

```

```

12         nn.Linear(hidden, channels),
13         nn.Sigmoid()
14     )
15
16     def forward(self, x):
17         b, c, _, _ = x.shape
18
19         s = self.pool(x).view(b, c)
20         s = self.fc(s).view(b, c, 1, 1)
21
22         return x * s

```

The essential operation is

$$Y = X \odot s.$$

This answers

Which channels are important?

32 Spatial Attention

Spatial attention asks

Where is the important information?

A simplified spatial attention mechanism can produce

$$A \in \mathbb{R}^{1 \times H \times W}$$

and compute

$$Y = A \odot X.$$

Thus channel attention asks

What?

while spatial attention asks

Where?

33 CBAM

CBAM combines

Channel Attention $\rightarrow$ Spatial Attention.

The feature transformation can be written conceptually as

$$X \rightarrow M_c(X) \odot X \rightarrow M_s(X) \odot X.$$

This is an example of evolving CNNs by adding a new function to the composition.

34 CNN + Attention: Why?

A convolution with a 3×3 kernel mainly captures local relationships.

For an image,

$$\text{pixel}_{i,j}$$

interacts directly with nearby pixels.

However, some tasks require relationships between distant regions.

For example,

$$\text{head} \leftrightarrow \text{body}$$

may involve spatially separated regions.

Attention can provide more direct long-range interaction.

Therefore:

$$\boxed{CNN : \text{local inductive bias}}$$

and

$$\boxed{Attention : \text{flexible/global interaction.}}$$

35 Vision Transformer

Vision Transformer changes the representation strategy.

An image is divided into patches.

For an image of size

$$224 \times 224$$

and patch size

$$16 \times 16,$$

the number of patches is

$$\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196.$$

Each patch is flattened and projected into an embedding.

Thus

$$\boxed{Image \rightarrow Patches \rightarrow Tokens.}$$

The tokens are processed using self-attention.

The high-level pipeline is

$$Image \rightarrow Patch \text{ Embedding} \rightarrow Transformer \rightarrow Classifier.$$

36 Self-Attention

Given token representations, attention uses queries, keys, and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V.$$

This is fundamentally different from a fixed local convolution.

37 CNN versus Transformer

	CNN	Transformer
Basic operation	Convolution	Self-attention
Interaction	Local	Potentially global
Inductive bias	Strong spatial locality	Weaker locality bias
Weight sharing	Kernel sharing	Projection sharing
Representation	Feature maps	Token sequences
Typical strength	Local visual structure	Long-range relationships

The comparison should not be interpreted as “one is always better”. Modern vision systems frequently combine both ideas.

38 A Unified View of CNN Evolution

The evolution can be summarized as

Neuron → Linear function
 → Activation
 → Layer
 → Function composition
 → Convolution
 → Deep CNN
 → Multi-scale CNN
 → Residual CNN
 → Efficient CNN
 → Attention CNN
 → CNN–Transformer
 → Vision Transformer.

Each step can be interpreted as adding a function that addresses a limitation of the previous representation.

39 Problem–Solution Interpretation

Problem	Operation	Purpose
Only linear transformations	Activation	Nonlinearity
Too many image parameters	Convolution	Locality + sharing
Large spatial size	Pooling/stride	Downsampling
Need hierarchical features	Deep stacking	Representation hierarchy
Very deep optimization	Residual connection	Easier information/gradient flow
Need feature reuse	Dense connection	Information propagation
Different object sizes	Multi-scale branches	Scale diversity
High computation	Depthwise convolution	Efficiency
Unequal feature importance	Attention	Selective weighting
Long-range relationships	Self-attention	Global interaction
Dense pixel prediction	Encoder-decoder + skip	Spatial reconstruction

40 Reading CNN Code Systematically

When reading a PyTorch model, students should ask the following questions.

Question 1: What is the input shape?

For example,

$$B \times 3 \times 224 \times 224.$$

Question 2: What function is applied first?

For example,

```
1 nn.Conv2d(3, 32, 3, padding=1)
```

means

3 channels $\rightarrow$ 32 channels.

Question 3: Does spatial size change?

Check

$$K, \quad P, \quad S.$$

Question 4: Is there nonlinearity?

Look for

```
1 nn.ReLU()
2 nn.GELU()
3 nn.SiLU()
```

Question 5: Is information reduced or preserved?

Look for

```
1 MaxPool2d
2 AdaptiveAvgPool2d
3 stride=2
```

Question 6: Is there a shortcut?

Look for

```
1 out = out + identity
```

This usually indicates a residual structure.

Question 7: Is there concatenation?

Look for

```
1 torch.cat(...)
```

This may indicate dense connections or multi-branch architectures.

Question 8: Is attention present?

Look for operations producing weights and multiplying features:

```
1 attention = ...
2 x = x * attention
```

or token-to-token interaction.

41 A Complete Educational CNN

The following model intentionally combines the fundamental ideas without using a complicated modern architecture.

```
1 import torch
2 import torch.nn as nn
3
4 class EducationalCNN(nn.Module):
5
6     def __init__(self, num_classes=10):
7         super().__init__()
8
9         self.block1 = nn.Sequential(
10             nn.Conv2d(3, 32, kernel_size=3, padding=1),
11             nn.BatchNorm2d(32),
12             nn.ReLU(),
13             nn.MaxPool2d(2)
14         )
15
16         self.block2 = nn.Sequential(
17             nn.Conv2d(32, 64, kernel_size=3, padding=1),
18             nn.BatchNorm2d(64),
19             nn.ReLU(),
20             nn.MaxPool2d(2)
21         )
```

```

22
23     self.block3 = nn.Sequential(
24         nn.Conv2d(64, 128, kernel_size=3, padding=1),
25         nn.BatchNorm2d(128),
26         nn.ReLU(),
27         nn.MaxPool2d(2)
28     )
29
30     self.head = nn.Sequential(
31         nn.AdaptiveAvgPool2d(1),
32         nn.Flatten(),
33         nn.Linear(128, num_classes)
34     )
35
36     def forward(self, x):
37         x = self.block1(x)
38         x = self.block2(x)
39         x = self.block3(x)
40         x = self.head(x)
41         return x

```

For

$$X \in \mathbb{R}^{B \times 3 \times 224 \times 224},$$

the spatial sizes are approximately

$$224 \rightarrow 112 \rightarrow 56 \rightarrow 28.$$

The adaptive average pooling then produces

$$128 \times 1 \times 1.$$

Thus the classifier receives only 128 values.

This is a useful example because the student can trace every operation.

42 Experiment: Inspecting Shapes

A very useful educational technique is to print tensor shapes.

```

1  model = EducationalCNN(num_classes=10)
2
3  x = torch.randn(2, 3, 224, 224)
4
5  print("Input:", x.shape)
6
7  x = model.block1(x)
8  print("After block1:", x.shape)
9
10 x = model.block2(x)
11 print("After block2:", x.shape)
12
13 x = model.block3(x)
14 print("After block3:", x.shape)
15
16 x = model.head(x)
17 print("Output:", x.shape)

```

Students should be able to predict the output shapes before executing the program.

That ability is one of the best tests of whether they understand CNNs.

43 Parameter Counting

For a convolution layer,

$$N_{\text{params}} = C_{\text{out}}C_{\text{in}}K_hK_w + C_{\text{out}}$$

if bias is used.

For

$$\text{Conv2d}(3, 32, 3),$$

we have

$$32 \cdot 3 \cdot 3 \cdot 3 + 32 = 896.$$

For a linear layer,

$$N_{\text{params}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}}.$$

This is important because architecture design is also a problem of computational resources.

44 Exercise: Build a CNN from Scratch

Students should implement the following function composition:

$$f = f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$$

where

$$f_1 = \text{Conv2D},$$

$$f_2 = \text{ReLU},$$

$$f_3 = \text{MaxPool},$$

$$f_4 = \text{Conv2D},$$

$$f_5 = \text{Classifier}.$$

A suggested exercise:

1. use MNIST or CIFAR-10;
2. construct a CNN without using a pretrained model;
3. print all intermediate tensor shapes;
4. calculate the number of parameters manually;
5. compare manual calculations with PyTorch;
6. train for several epochs;
7. plot training and validation loss;
8. inspect several incorrect predictions.

45 Exercise: Compare CNN Variants

Create four models:

$$M_1 : \textit{Conv} + \textit{ReLU} + \textit{Pool},$$

$$M_2 : M_1 + \textit{BatchNorm},$$

$$M_3 : M_2 + \textit{Residual},$$

$$M_4 : M_3 + \textit{Attention}.$$

The objective is not merely to obtain the highest accuracy.
Students should compare

accuracy + parameters + training time + representation behavior.

This makes the architectural evolution experimentally meaningful.

46 A Practical Mental Model

When encountering any CNN architecture, ask:

1. What is the input representation?
2. What local operation is being learned?
3. Where is nonlinearity introduced?
4. How does spatial resolution change?
5. How does the number of channels change?
6. Is information added, discarded, or reused?
7. Are there shortcut paths?
8. Are some features weighted more strongly?
9. Are interactions local or global?
10. What limitation of an earlier architecture is being addressed?

These ten questions are more useful than memorizing the names of dozens of architectures.

47 Final Conceptual Map

The complete conceptual map is

MATHEMATICAL FUNCTION

$$f(x)$$

↓

NEURON $z = w^T x + b$

↓

NONLINEAR NEURON $h = \sigma(w^T x + b)$



LAYER $h = f_{\theta}(x)$



DEEP NETWORK $F = f_L \circ \dots \circ f_1$



CNN *Conv* + Activation + Downsampling



DEEP CNN



RESIDUAL / DENSE / MULTI-SCALE



ATTENTION



CNN-TRANSFORMER / VISION TRANSFORMER

The central message is therefore:

A neural architecture is a structured composition of functions.

And the evolution of architectures can be understood as

new architecture = old architecture + a mechanism addressing a limitation.

This viewpoint allows students to move from simply “using CNN” to actually understanding, analyzing, and designing neural networks.

Suggested Laboratory Sequence

1. **Lab 1:** Implement a neuron using NumPy.
2. **Lab 2:** Implement linear regression as function composition.
3. **Lab 3:** Add ReLU and build a multilayer perceptron.
4. **Lab 4:** Implement a small convolution using NumPy.
5. **Lab 5:** Build the first CNN using PyTorch.
6. **Lab 6:** Calculate tensor shapes and parameters manually.
7. **Lab 7:** Compare pooling and stride.
8. **Lab 8:** Build a residual block.
9. **Lab 9:** Add channel attention.
10. **Lab 10:** Compare CNN, ResNet-style CNN, and attention CNN.
11. **Lab 11:** Study a pretrained modern CNN and map every component to the functions learned in this tutorial.
12. **Lab 12:** Compare a CNN representation with a Transformer representation.